UMHACKATHON 2026 · DOMAIN 1 · AI SYSTEMS & AGENTIC WORKFLOW AUTOMATION

Product Requirements.

The product the platform is, the users it serves, the scope it commits to for the UMHackathon 2026 submission.

TEAM · OPUS MAGNUM

Oybek Odilov
Al-Bazeli Gameel Abduljalil
Lee Yih Shen
Samandar Erkinjonov
Abdugaffor Odilov

ONBOARD

POWERED BY Z.AI GLM · REGULATED ONBOARDING ENGINE

PRD · PRODUCT REQUIREMENTS APRIL 2026 SUBMISSION RELEASE

1. Problem Definition & Purpose

Opening a new building in Malaysia is slow, paper-heavy, and local. Ownership certificates, fire-safety approvals, zoning clearances, floor plans, safety reports — each submitted to a different office, each with its own format, each city wanting it slightly differently. Developers file paperwork they do not fully understand and wait weeks for feedback; cities receive a flood of non-standard submissions; the public has no good way to discover what is new, approved, or available.

The bottleneck is not bureaucracy as a concept. It is the **absence of a system that can read what's submitted, check it against what a city requires, and keep the process moving without waiting on a human to re-read every form**.

Purpose. Opus Magnum turns regulated building onboarding into a single guided workflow, powered end-to-end by an AI that reads documents, verifies them against city-specific criteria, reconciles contradictions, *proposes corrections*, and publishes the building as a public, explorable listing.

Larger ambition. Buildings are the concrete demo. The engine underneath is a **general-purpose adaptive, multi-modal, AI-verified onboarding platform** — the same template model applies to supplier qualification, license renewal, KYC, regulated-professional registration. We ship buildings because the pain is concrete and visual; the pattern is reusable.

2. Target Users & User Stories

Three roles, one engine, one knowledge base.

2.1 Admin — the city's configuration point

Who. City-office workflow owner (municipal digital-services manager, or equivalent). **Core job.** Define the onboarding process a city requires: which forms, which documents, which verification criteria, in what order.

Stories.

- *As an admin, I can describe my city's requirements in one sentence and get a GLM-drafted starter template I can edit, so I don't start from a blank page.*
- *As an admin, I can ask a tutor assistant to explain why each step exists and what a field should capture, so I can author confidently without a manual.*
- *As an admin, I can publish a template and have every new onboarding in my city run through that exact sequence, so submissions become comparable across buildings.*

2.2 Building owner — the workflow user

Who. Developer, property-management lead, or their appointed submitter. **Core job.** Onboard a specific building by running that city's template to completion.

Stories.

- *As an owner, I can see the whole onboarding process as a live graph, so I always know what is done, what is running, and what is waiting on me.*
- *As an owner, when I upload a compliance document, I can watch the AI process it in real time — received, parsed, extracted, verified — and see a plain-language pass or fail verdict with cited evidence.*
- *As an owner, when I upload a regulator document (a deed, a Bomba certificate, an MBSA letter) the AI autofills the relevant form fields and asks me to confirm, so I'm not retyping facts the document already states.*
- *As an owner, when the AI flags a contradiction between documents, I can see exactly which values disagree and either correct them or accept the AI's proposed fix without a round-trip to a reviewer.*
- *As an owner, when something goes wrong mid-run, I can rewind to a specific step and re-run from there without losing earlier state.*
- *As an owner, when all steps pass, my building is published automatically — the paperwork and the public listing come from the same submission.*

2.3 Visitor — the public audience

Who. Anyone curious about a published building — prospective tenants, journalists, city residents, neighbouring businesses. **Core job.** Discover a building and ask grounded questions about it.

Stories.

- *As a visitor, I can browse a directory of published buildings and open one to see a 3D view and a chat panel.*
- *As a visitor, I can ask the assistant anything about the building, and it answers only from what the owner submitted — with citations to the source profile field, never invention.*
- *As a visitor, when I ask "show me the rooftop," the assistant nudges the camera in the 3D scene rather than asking for a photo it does not have.*

3. Originality, Innovation & Value Realisation

3.1 What is original

The workflow is data, not code. A new city is a new JSON template stored in Postgres. The same engine runs every template. There is no per-city codebase, no YAML rule files, no "compliance-rules microservice". The admin's template *is* the rule set.

Compliance docs vs content docs are explicit first-class types. Compliance docs feed pass/fail verdicts and are never shown publicly. Content docs feed the building's profile and ground the visitor assistant. The same upload workflow handles both, the system never conflates them, and the owner submits everything once.

Autofill from regulator documents. Uploading a deed or a Bomba certificate doesn't just verify — it pre-populates the form fields the document already states, with the AI proposing a corrected value when the form and the document disagree. The owner confirms or overrides. This is what closes the "but I already wrote this in the deed" complaint that drives most resubmissions.

Every GLM call is logged for audit. Each step's `decision_log` captures the prompt, the tool called, the model's arguments, and the duration. Long content blocks are elided so the log stays scannable. This is the drift net for AI behaviour and the substrate for human-in-the-loop review of AI judgements.

3.2 Why it matters

- **For admins:** a city that wants to onboard buildings for the first time can be live in minutes, not quarters. The marginal cost of a new jurisdiction is one template.
- **For owners:** the feedback loop collapses from weeks to seconds. A rejection cites the exact reason; a contradiction shows exactly which values disagree; corrections go straight back into the graph.
- **For cities:** submissions become uniform and auditable. Every decision trail is preserved.
- **For the public:** discovery becomes first-class. A building's page is not a document dump — it is a conversable, grounded listing.

3.3 The "AI at the centre" invariant

If GLM is removed, six things stop working:

1. **Template drafting.** Admins lose the GLM-generated starter template and the tutor assistant; only manual authoring remains.
2. **Document parsing & fact extraction.** Raw PDFs / images never become structured fields.

3. **Compliance verification.** Plain-English criteria cannot be applied to a document without reading it.
4. **Cross-document reconciliation.** Contradiction surfacing requires reasoning over multiple extracted outputs.
5. **Auto-fix proposals.** The owner stops getting AI-suggested corrections — the system reverts to "type it in yourself, look up the rules".
6. **Visitor assistant.** Every grounded answer depends on reading the building's content documents.

Form rendering, file storage, and the React canvas survive. Nothing else does. This is the architectural litmus test.

4. Feature Prioritisation & MVP Scope

4.1 In scope for MVP

AREA	FEATURE	STATUS
Admin	City CRUD	Shipped
Admin	Template editor with primitive-specific config	Shipped
Admin	`draft_template` (GLM drafts full template from one-line brief)	Shipped
Admin	Template assistant (`explain_field` , SSE-streamed)	Shipped
Owner	Building create + city selection	Shipped
Owner	Workflow canvas (custom DAG renderer) with live step status	Shipped
Owner	Dynamic form renderer for `collect_form` (with index-fallback when field name is missing)	Shipped
Owner	Drag-drop upload with live per-file processing cards	Shipped
Owner	Compliance verification loop (re-upload on fail)	Shipped
Owner	**Autofill from regulator documents** (`extract_document` → `propose_auto_fix`)	Shipped
Owner	Content extraction into the building profile	Shipped
Owner	Cross-document check with severity grading and trivial-diff exclusions	Shipped
Owner	Human-review summary + publish	Shipped
Owner	**Rewind to step** (run resumes from a chosen earlier step)	Shipped
Owner	Per-step decision-log drawer (every GLM call visible)	Shipped
Visitor	Published directory	Shipped
Visitor	3D scene (real GLB apartment models + scene_config overlay)	Shipped
Visitor	Grounded assistant chat (SSE-streamed; profile + content docs; camera-nav cues)	Shipped
Platform	Six workflow primitives, composable per city	Shipped

AREA	FEATURE	STATUS
Platform	Eight GLM tools (see §5.1)	Shipped
Platform	LangGraph `StateGraph` per run with SQLite/Postgres checkpointer	Shipped
Platform	OpenAI-SDK integration with ILMU `ilmu-glm-5.1`	Shipped
Platform	`decision_log` persisted per step (with content-block elision over 4k chars)	Shipped

4.2 Out of scope — flagged explicitly

- **Real auth.** Mocked session: one seeded admin and one seeded owner. Drop-in JWT is the designed upgrade path; this was cut to focus engineering on the GLM-critical loop.
- **Payments / transactions.** Not in the onboarding value chain for MVP.
- **Multi-tenant isolation & row-level security.** Single-tenant demo.
- **Production observability & alerting.** Dev-grade structured logging only.
- **Second jurisdiction.** The engine is config-driven; only Shah Alam is authored end-to-end. Adding Kuala Lumpur would be one JSON file.
- **Vision documents.** `ilmu-glm-5.1` does not accept image inputs (verified empirically). Image documents go through OCR (`pytesseract`) before GLM sees them.
- **Embeddings / RAG.** Visitor assistant grounds on raw profile + content-doc text inline; GLM's 200k context absorbs single-building corpora without an index.

4.3 Prioritisation rationale

The demo judged here is three minutes long. Every shipped feature earns a beat in that demo. Features that do not appear in the demo script (real auth, payments, observability) were cut so the GLM-critical path could be built deeply rather than broadly.

5. AI Model & Prompt Design

5.1 Model selection

The platform's reasoning engine is **`ilmu-glm-5.1`** via the ILMU API (`https://api.ilmu.ai/v1`). ILMU is OpenAI-compatible, so we use the official `openai` Python SDK with a swapped `base_url` . Three properties make GLM 5.1 the right fit for an AI-centric onboarding engine:

- **Long context (200k tokens).** Whole content-document corpora for a single building fit inline, so the visitor assistant can ground answers without an embeddings layer for the MVP.

- **Native tool use + JSON mode.** Every primitive that needs reasoning is exposed as a typed function call (eight tools in total — see §5.2) and returns parsed arguments, eliminating brittle text parsing.
- **Reasoning-effort knob.** `reasoning_effort="low"` is materially faster on extraction and verification; free-form tutor and visitor streams omit the parameter so the model can think when grounding requires it.

Using any other reasoning model is explicitly disqualifying for this domain; GLM is therefore both a product requirement and a hackathon constraint. The OpenAI SDK is the *transport*; GLM is the *brain*. The client wrapper (`backend/app/glm/client.py`) keeps the provider configurable for local-dev convenience (e.g., when GLM is rate-limited during a debugging session), but the submission, the seed data, the demo, and every test fixture run end-to-end on `ilmu-glm-5.1`.

5.2 Prompting strategy

Every reasoning step is **single-shot, tool-forced, structured-output**. We don't run multi-turn chains for the verification path:

- Each primitive issues exactly one `chat.completions.create` with `tool_choice` set to the relevant function and a tightly scoped system prompt.
- The tool schema (in `backend/app/glm/tools.py`) is the contract: GLM returns parsed arguments, the primitive validates them, the run advances. No chain-of-thought parsing, no "did the model say yes" string matching.

The eight tools, each with its own narrow system prompt:

1. `draft_template` — author a complete workflow from a plain-English city description (must end with `publish` , max nine steps, six allowed primitives).
2. `explain_field` — tutor an admin on what a step or field is for; clean markdown, no tables unless asked.
3. `extract_document` — OCR text → structured facts; skip unknowns, keep notes ≤ 2 sentences.
4. `verify_against_criteria` — pass / fail / unclear per criterion, with cited evidence.
5. `cross_check_documents` — reconcile structured outputs across documents with explicit exclusions for trivial diffs (date formats, certificate issue dates ≠ year-built, office addresses).
6. `summarize_for_review` — pre-publish summary in markdown with a concrete gap list.
7. `answer_visitor_question` — building concierge grounded in profile + content excerpts; scene-nav requests are camera cues, not "send me a photo" replies; ≤ 4 sentences.
8. `propose_auto_fix` — suggest form corrections from regulator-issued documents; treat deeds, Bomba certs, and MBSA letters as authoritative; never propose changes to uploaded documents.

The free-form admin tutor and visitor chat are **multi-turn**: SSE-streamed dialogue with conversation history and a domain-grounding payload. The system prompt enforces "answer only from the grounding material; otherwise say you do not know."

5.3 Context and input handling

- **Compliance documents** flow through `processing/pipeline.py` : `pypdf` extracts the text layer; `pdf2image + pytesseract` OCR is the fallback for scanned PDFs. Images route directly to `pytesseract` . GLM never sees raw PDFs because `ilmu-glm-5.1` lacks vision.
- **Per-document budget.** Extracted text is truncated to ~30,000 characters before reaching `extract_document` ; verification works on the structured output, not the raw text, so this ceiling never starves the criteria check.
- **Visitor grounding budget.** The assistant assembles `building.profile + scene_config + content-doc extracts` and caps the grounding payload at ≤ 60,000 characters (well inside GLM's 200k window). Anything longer is summarised oldest-first.
- **Hard rejections.** Empty extractions (OCR returned 0 chars) are surfaced as "document unreadable — please re-upload as searchable PDF" *before* any GLM call is made. We never feed empty payloads to the model.
- **Decision-log content elision.** Long bodies in the audit log are truncated past 4k characters and image data URIs are stripped. The log stays scannable; we never re-feed entire documents back into prompts during retries.

5.4 Fallback and failure behaviour

- **Transient API errors (429 / 5xx).** `GLMClient._chat` retries up to 4 times with exponential backoff and jitter.
- **Malformed tool arguments.** `call_tool` catches `json.JSONDecodeError` and returns `{_error, _raw}` ; the primitive marks the step `failed` with a retry affordance instead of crashing the run.
- **Hallucinated or off-topic completions.** Verification compares structured arguments against the criteria list — `"verdict": "pass"` with no cited evidence is rejected and re-prompted once; on a second failure the step is marked `failed` for human review.
- **Visitor grounding miss.** The system prompt forces "I don't know based on the documents this owner submitted" when the grounding lacks the answer. We do not fall back to general knowledge.
- **Owner-driven rewind.** A failed step is not terminal — `POST /buildings/{id}/run/rewind/{step_id}` bumps `RunState.rev` , allocates a fresh checkpoint thread, and re-runs from the chosen step. State from earlier steps is preserved.

- **Human-review escalation.** Every primitive can route to `human_review`, which surfaces the failing step's `decision_log` and a GLM-written summary so a reviewer can override or restart.
- **Auditability.** Every call — successful or failed — writes to `step_runs.decision_log` with prompt, tool, arguments, response, and duration. Failures that escape review are forensically reproducible.

6. Success Metrics (MVP)

METRIC	TARGET	HOW MEASURED
Live GLM calls per run	≥ 12 per full Shah Alam template	`decision_log` count per `workflow_runs.id`
Compliance verification accuracy	≥ 9 / 10 on labelled fixtures	`tests/test_upload_compliance.py` + GLM eval set
Verification rejection with evidence	100 % of failing cases cite offending text	Decision-log inspection
End-to-end demo time	≤ 3 minutes live	Timed rehearsal
Template authoring time	≤ 2 minutes from blank to publish via `draft_template`	Manual measurement
Autofill correctness on seeded docs	≥ 8 / 10 fields auto-populated correctly	`propose_auto_fix` output vs hand-labelled gold

7. GLM-Centrality Validation

Every one of the six critical surfaces (template drafting, extraction, verification, cross-check, autofill, visitor chat) calls GLM at runtime, each logged under `step_runs.decision_log` with its tool name, arguments, and timing. A reviewer who stubs `GLMClient` to raise immediately would see the workflow freeze at step 1, template drafting fail, autofill silently no-op, and the visitor assistant return an error. The litmus test passes.

8. Assumptions & Constraints

8.1 LLM cost constraint

A typical Shah Alam onboarding fires ~15–25 GLM calls (one per `extract_document` per uploaded doc, one per `verify_against_criteria`, one per autofill, plus admin and visitor turns). With `reasoning_effort="low"` on extraction/verification, an end-to-end run lands around 40–80k input + 6–10k output tokens.

Two design decisions hold the cost line:

- **`reasoning_effort="low"` is the default on every structured-output path.** Extraction, verification, cross-check, and autofill don't benefit from deep reasoning; the model still emits the right schema, faster and cheaper.
- **Decision-log content elision.** Bodies past 4k characters are truncated so we never re-feed entire documents back into prompts during retries or human review.

A future cost lever (out of MVP scope) is response caching of `extract_document` keyed by file SHA — a re-uploaded duplicate would skip the model entirely.

8.2 Technical constraints

- **Vision unavailable on `ilmu-glm-5.1`.** Image documents must pass through OCR before the model sees them.
- **Auth is mocked.** A seeded admin and a seeded owner stand in for a real identity provider. Drop-in JWT is the designed upgrade path.
- **Single-tenant demo.** No row-level security or per-tenant data partitioning.
- **One jurisdiction is authored end-to-end.** The engine accepts any number of city templates; the submission ships Shah Alam. A second city is one JSON file.
- **No Alembic.** Schema bootstrap via `Base.metadata.create_all()`; reseeding goes through `app.seed.reseed_demo` which drops/recreates app tables and clears LangGraph checkpoint rows in place (safe with the backend running).

8.3 Performance constraints

- **GLM rate limits.** ILMU's published rate limits apply; the demo path has been profiled to stay below the per-minute quota at one concurrent run.
- **OCR throughput.** `pdf2image + pytesseract` on a multi-page scanned PDF can take 5–15 seconds; users see a per-file processing card with live status so latency is never silent.
- **SSE polling cadence.** The run-progress SSE endpoint polls the DB at 500 ms — adequate for one demo run, replaceable by a Redis pub/sub bus for production load.

8.4 User input

- **Owner-submitted documents are trusted as-is.** The system never modifies an uploaded file; it only extracts, verifies, audits, and proposes fixes to *forms*.
- **Required-field gating.** A `collect_form` step cannot advance until the owner submits the configured fields; the canvas highlights the awaiting node until input arrives.
- **Re-upload loop on failure.** A failed verification re-routes to the same upload step; `pending_input[step_id]` is cleared so the new upload kicks off a fresh GLM verification.
- **Human review is the final stop.** When `human_review` fires, no automated decision overrides the reviewer; the model's summary is advisory, not authoritative.

9. Risks & Mitigations

RISK	IMPACT	MITIGATION
`ilmu-glm-5.1` rate limits trigger during demo	Mid-demo stall	Exponential-backoff retry in `GLMClient._chat`; demo path uses ≤ 25 calls end-to-end
Vision not supported on GLM 5.1	Image docs can't be read directly	OCR-only fallback (`pytesseract`), flagged in decision log; baseline verified
GLM returns malformed tool arguments	Step stalls	`call_tool` catches `json.JSONDecodeError`, returns `{_error}` so the UI can show a retry affordance
Owner submits a scanned PDF with no text layer	Silent empty extract	`processing/pdf.py` falls back to `pdf2image` + `pytesseract`; decision log marks `ocr_used=true`
Template becomes invalid mid-edit (unknown primitive)	Run crashes	`compile_template` raises `ValueError("unknown primitive: …")` before the graph executes
Cross-check surfaces trivial differences (date formatting)	Owner gets noisy false-positive contradictions	Cross-check prompt has explicit exclusion rules; severity ≤ "minor" hidden from owner UI
Autofill suggests a wrong value with confidence	Owner accepts a bad fix	UI requires explicit confirmation before applying; original value is preserved in decision log
LangGraph checkpoint corruption	Run cannot resume	Reseed script clears checkpoint rows in place; rewind allocates a fresh thread per `RunState.rev`
OCR binary missing on the host	Image extraction silently empty	`processing/pdf.py` and `processing/image.py` log `ocr_used=false`; tests gate on the `[ocr]` extra

10. Future Work (not in this submission)

- Second city authored (Kuala Lumpur) to prove config-only expansion.
- Real multi-tenant auth + per-seat isolation.

- Embeddings-based RAG for very large content corpora (fallback if a single-building content corpus exceeds 200k tokens).
- Vision-capable model upgrade when ILMU adds one to our plan.
- City-side reviewer surface that spot-checks GLM verdicts.
- Response cache for `extract_document` keyed by file SHA.

Companion to the SAD (architecture as shipped), the TAD (technology rationale), and the QATD (test strategy).